Ten Queries

DB25's SQL front end, measured against PostgreSQL 18 and DuckDB 1.5 at the same boundary

September 2026 · DB25 project · `space-rf-org/db25`

A disclaimer, because it should come first

We do not chase numbers. This one is for fun.

Query-planner benchmarks are the easiest numbers in databases to make say whatever you want. Choose the fixture, choose the queries, choose where to start and stop the clock, and any system can be made to win. Nobody should make a decision from what follows. It is ten queries against a 192,000-row fixture on one 4-core VM. It is not a workload, it is not TPC anything, and DB25 does not yet have an executor, so it cannot run a workload even if we had one.

So why publish it?

Because a query planner is one of the few parts of a database whose quality can be inspected without running anything. You can read what it chose and what it believed, line by line, and hold both against systems that have been getting it right for thirty years. That comparison stays meaningful even where the timings do not, and it is worth doing in public.

We also publish the numbers that make us look bad, and the two occasions on which this paper reported a result that was not there and had to withdraw it. Section 7 is where DB25 loses; section 9 is where the method did.

---

1. The boundary, and why it is the honest one

DB25 is a SQL front end: tokenizer → parser → semantic analyzer → binder → logical plan → optimizer → **physical plan**. There is no execution engine. So the only comparison that is not a lie is one where PostgreSQL and DuckDB are stopped at exactly the same place: SQL text in, chosen physical plan out, executor never started.

Both of them can be stopped there:

system	how it is stopped	what is included
DB25	`lower()` returns the physical plan	tokenize, parse, analyze, bind, logical optimize, Cascades search
PostgreSQL 18.6	`EXPLAIN` without `ANALYZE`	parse, rewrite, plan (two numbers — see below)
DuckDB 1.5.5	in-process `EXPLAIN`	parse, bind, optimize, physical plan

PostgreSQL is reported two ways because its own `Planning Time` **excludes raw parse and rewrite**, while DB25's number includes parse. Reporting only `Planning Time` would flatter us; reporting only wall time would

flatter PostgreSQL by loading it with protocol overhead. Both are in the table. Where they disagree, believe the one that is worse for us.

This boundary is not neutral and we are not pretending it is. It favours DB25 in one way — everything is in-process, with no client protocol and no catalog round trip — and favours the others in another — they have real statistics, which DB25 does not, and statistics cost time to consult. Section 8 lists the rest.

Machine and versions

4 × Intel Xeon @ 2.10 GHz, 15 GB RAM, Linux 6.18, gcc/g++ 13.3, everything at `-O2`. All three systems measured back to back in one session on an otherwise idle host, with identical fixtures and `ANALYZE` run on both engines.

- **DB25** at `673e71c`, Release build.
- **PostgreSQL 18.6**, built from source at `REL_18_STABLE` with `-O2`, `--without-icu`, C locale. Stated because it matters: this is not a distribution build, and a distribution build with ICU, readline and a UTF-8 collation will not produce identical microseconds. The estimates it produces are unaffected by any of that; the timings are its own build's.
- **DuckDB 1.5.5**, the released Python wheel, in-process.

Fixture

table	rows	shape
`items`	100,000	fact; FKs to `orders`, `prod`
`orders`	50,000	FK to `cust`
`emp`	20,000	FK to `dept`
`cust`	20,000	
`prod`	2,000	
`dept`	50	

Every join predicate in every query is a foreign key onto a primary key. That is deliberate: it is the case where the right answer is *knowable*, so a wrong estimate has nowhere to hide.

All three systems are given statistics. PostgreSQL and DuckDB had `ANALYZE` run. DB25's catalog is caller-seeded with the fixture's real distributions — per column: distinct values, null fraction, numeric range, and whether the column is unique. Those are the four numbers its estimator reads; it has no histogram and no most-common-value list. Seeding them by hand rather than measuring them is exactly what a planner with no engine must do, and section 8 records it as the limitation it is.

---

2. The ten queries, verbatim

```
-- q1  filter
SELECT name FROM emp WHERE salary > 1000;

-- q2  two-way join
SELECT e.name, d.name FROM emp e JOIN dept d ON e.dept_id = d.id;

-- q3  four-way join
SELECT c.name FROM cust c
  JOIN orders o ON c.id = o.user_id
  JOIN items  i ON o.id = i.order_id
  JOIN prod   p ON i.prod_id = p.id;

-- q4  six-way join
SELECT c.name FROM cust c
  JOIN orders o ON c.id = o.user_id
  JOIN items  i ON o.id = i.order_id
  JOIN prod   p ON i.prod_id = p.id
  JOIN emp    e ON e.id = c.id
  JOIN dept   d ON d.id = e.dept_id;

-- q5  group by + aggregates
SELECT dept_id, COUNT(*), SUM(salary) FROM emp GROUP BY dept_id;

-- q6  having
SELECT dept_id, SUM(salary) FROM emp GROUP BY dept_id HAVING SUM(salary) > 100;

-- q7  window function
SELECT name, RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) FROM emp;

-- q8  correlated EXISTS
SELECT e.name FROM emp e
 WHERE EXISTS (SELECT 1 FROM orders o WHERE o.user_id = e.id);

-- q9  CTE + join + non-equi condition
WITH s AS (SELECT dept_id, AVG(salary) AS a FROM emp GROUP BY dept_id)
SELECT e.name FROM emp e JOIN s ON e.dept_id = s.dept_id
 WHERE e.salary > s.a;
```

And the one we care most about, because it is the one that exercises everything at once — CTE, aggregate, window function, inner join, `LEFT JOIN LATERAL` with a correlated scalar aggregate, a non-equi filter, an `ORDER BY` over a window output, and a `LIMIT`:

```
-- q10  reference query
WITH dept_stats AS (
  SELECT dept_id, AVG(salary) AS avg_sal, COUNT(*) AS headcount
    FROM emp GROUP BY dept_id
)
SELECT e.name,
       e.salary,
       ds.avg_sal,
       RANK() OVER (PARTITION BY e.dept_id ORDER BY e.salary DESC) AS rnk,
       ord.total
  FROM emp e
  JOIN dept_stats ds ON e.dept_id = ds.dept_id
  LEFT JOIN LATERAL (
        SELECT SUM(o.total) AS total FROM orders o WHERE o.user_id = e.id
  ) ord ON true
 WHERE e.salary > ds.avg_sal
 ORDER BY rnk
 LIMIT 10;
```

`LATERAL` is the interesting part. Its right side references `e.id` from the left, so it cannot be evaluated standalone: the only admissible implementation is a nested loop that re-evaluates the subquery per left row. DB25's applicability rules say so structurally rather than discovering it by costing — a hash join there would not merely be slower, it would not be executable. The plan in Appendix B shows it choosing `NestedLoopJoin kind=leftlateral` with an `(outer d1 #0)` correlation reference, which is the front end getting a genuinely hard construct right.

---

3. Planning time, and what this host can support

What DB25's number contains

DB25's figure is the **whole front end**, parser included. PostgreSQL's own `Planning Time` is not — it measures `pg_plan_query` alone, after parse analysis and rewrite have already run. Comparing the two directly charges DB25 for work PostgreSQL is not being charged for, so the stages are broken out and mapped onto each other:

DB25 stage	PostgreSQL equivalent	in `Planning Time`?
`parse`	`raw_parser`	no
`analyze` + `bind`	parse analysis + rewrite	no
logical optimize + physical lowering	the planner	**yes**

The honest precision of this section

Absolute microseconds on this host are good to about a factor, not a percent. Nine repeats of the same binary on the same query give totals that are not noisy around a mean but *bimodal* — q1 lands on either 6.4 µs or 10.8 µs, and the mode does not depend on anything the program does. Pinning to one core narrows it and does not remove it. This is a shared cloud VM; the neighbours are not ours to quiet.

So the absolute table below carries its observed range, and **differences under about 2× are not claims**. What survives that rule is stated after it, and it is less than an earlier draft of this paper asserted.

What *does* survive cleanly is any number that is a **ratio within one run**, because host drift cancels. The stage shares below moved by at most 2.8 percentage points across nine repeats while the absolutes they are computed from swung by 50–90%.

Where the time goes — stable

	query	parse	analyze+bind	planner
q1	filter	17%	27%	56%
q2	2-way join	19%	25%	56%
q3	4-way join	9%	13%	**78%**
q4	6-way join	9%	14%	**76%**
q5	group by	18%	34%	48%
q6	having	18%	31%	51%
q7	window	21%	25%	54%
q8	EXISTS	21%	28%	51%
q9	CTE + join	18%	27%	55%
q10	reference	16%	28%	55%

Parsing is **9–21%** of the front end and never the story. The planner is **49–78%**, and its share rises with join count — which is what it should do, and is the only part of this section that is really about planning.

Absolute times — read with the range

Medians of nine repeats (DB25), five (PostgreSQL), three (DuckDB), µs. The bracketed figure is the observed spread as a percentage of the median.

	query	DB25 planner	DB25 total	PG 18 planner	PG 18 parse+plan	DuckDB
q1	filter	3.6	6.5 [50%]	4.0	7.5	248
q2	2-way join	6.3	11.3 [2%]	12.0	29.4	485
q3	4-way join	35.9	46.1 [2%]	44.0	93.0	1126
q4	6-way join	**55.8**	72.8 [30%]	158.0	271.0	1945
q5	group by	4.9	10.1 [59%]	6.0	22.8	430
q6	having	6.2	12.2 [57%]	7.0	26.5	594
q7	window	5.0	9.2 [63%]	6.0	22.0	299
q8	EXISTS	5.8	11.4 [55%]	17.0	37.1	571
q9	CTE + join	12.4	22.7 [57%]	22.0	61.5	1302
q10	reference	34.2	62.5 [90%]	63.5	201.5	1873

What survives the 2× rule. DuckDB is 24–57× slower than DB25 end to end and 12–85× slower than PostgreSQL's planner: that gap is far outside the noise. So is the six-way join, where DB25's planner is about 2.8× faster than PostgreSQL's (55.8 against 158.0) — the one per-query planning-time claim this host supports.

Everything else is a tie. An earlier draft of this paper reported DB25 ahead on four queries and behind on four others; neither of those lists survives a measurement that varies by 50% between runs of the same binary. They are recorded as indistinguishable, which is what they are.

DuckDB's 24–57× is not a defect in DuckDB. It is an analytical engine whose planning cost is amortised over multi-second scans, and section 4 shows what it spends the time on — 33 optimizer passes including statistics propagation and a sampling join-order search. It is in the table because it is a fair third data point at this boundary, not because 2 ms is a problem for anyone running DuckDB.

The framing this section earns: planning time is not why DB25 exists, and at execution scale it is noise. Section 4 is the section that matters, and it has the advantage of being exactly reproducible — an estimate is a pure function of the plan and the statistics, so the same input gives the same answer on every run, on any host. None of the uncertainty above touches it.

---

4. Cardinality estimates — the numbers that actually matter

A planner's speed is a convenience. Its **estimates** are what decide the plan, and a plan chosen from a wrong estimate is slow no matter how fast it was chosen.

These numbers have a property section 3's do not: they are **exactly reproducible**. An estimate is a pure function of the plan and the statistics, so the same query gives the same answer on every run and on any host. Nine repeats of DB25 produced bit-identical output.

Top-node estimated rows against the true result size:

	actual	DB25	PostgreSQL 18	DuckDB
q1 filter	18,999	**19,000**	**18,999**	4,000
q2 2-way join	19,600	**20,000**	**20,000**	22,222
q3 4-way join	99,950	**100,000**	**100,000**	26,961
q4 6-way join	98,000	**100,000**	**100,000**	15,695
q5 group by	50	**50**	**50**	45
q6 having	50	17	17	9
q7 window	20,000	**20,000**	**20,000**	**20,000**
q8 EXISTS	19,999	**20,000**	19,739	4,000
q9 CTE + join	10,000	6,666	6,667	663
q10 reference	10	**10**	**10**	5,000

Aggregated as q-error — the geometric mean of `max(est/act, act/est)`, the standard measure since Leis et al.'s *How Good Are Query Optimizers, Really?*:

system	geometric mean q-error	statistics consulted
DB25	**1.16×**	ndv, null fraction, column range, uniqueness
PostgreSQL 18	1.17×	histograms, n_distinct, MCVs, correlation
DuckDB 1.5	5.58×	full, with `ANALYZE` run

How it got there

Three increments, each measurable on its own:

	geometric mean q-error
flat constants (one selectivity for every predicate)	3.09×
+ per-column statistics for filters and `GROUP BY`	1.47×
+ joins and semi joins reading their keys' distinct counts	**1.16×**

The flat model's errors were concentrated and large: `WHERE salary > 1000` kept 10% of its rows where the truth is 95%, and `GROUP BY dept_id` produced one group per ten input rows where the truth is fifty groups out of twenty thousand rows. Neither is a rule that a better rule fixes. A filter's selectivity is a property of the data, and only a statistic estimates it.

The tie, and why it is the interesting part

1.16× against 1.17× is a tie, not a win. Reading it as a win would be exactly the number-chasing this paper opened by disclaiming — the two differ by one row on one query.

What is worth reporting is *why* they tie. After the statistics work, DB25 and PostgreSQL are exactly right on eight of the ten queries, and wrong on the same two, by the same amounts:

	actual	DB25	PostgreSQL	what neither can estimate
q6 having	50	17	17	a filter over an **aggregate's output**
q9 CTE + join	10,000	6,666	6,667	a column against **its own group mean**

Both fall back to the Selinger default of one third for an inequality that cannot be estimated, and both land in the same place. No histogram carries a correlation between a column and an aggregate computed over it; this is not a gap in either catalog, it is a question statistics of this kind do not answer.

DB25 reaches that position consulting four numbers per column — distinct values, null fraction, numeric range, and a uniqueness flag — and no histogram or most-common-value list at all. That is the claim worth making, and it is a claim about how far simple statistics go, not about beating anyone.

What statistics cost

Nothing measurable. Interleaved A/B over the ten queries, five pairs each, gives a **median planning-time delta of 0.0%**, with nine of ten inside ±2.5%. A predicate tree walk and a handful of hash lookups do not register against tens of microseconds of search.

One honest note on how that was established, because it nearly went the other way. The first A/B showed q1 **32% faster with statistics**, consistently across five pairs. It was not an effect: the candidate counts were identical, and the *parse* stage — which statistics cannot touch — was 35% faster too. q1 is bimodal on this host, and the two arms had sampled different modes. The control that caught it was running the same setting six times and seeing both modes appear within it.

---

5. Search shape

What DB25 is actually doing, per query:

	memo groups	candidates	goals	pruned
q1 filter	3	3	3	0
q2 2-way join	4	7	5	1
q3 4-way join	11	45	23	18
q4 6-way join	16	63	31	25
q8 EXISTS	4	5	4	1
q9 CTE + join	7	11	10	1
q10 reference	15	20	23	2

Goals exceeding groups is the point of the whole design. A goal is a (group, required-properties) pair: the same logical subtree optimized more than once, for different demands from above. Bottom-up planners cannot ask that question — they settle the join before the `ORDER BY` is considered and then sort whatever came out. q4 optimizes 16 groups for 31 goals and q10 optimizes 15 for 23, which is the top-down property-directed search visibly doing what it exists to do. q1, q5 and q7 have goals equal to groups, which is also correct: nothing in them demands a property from below.

These counts are the same with statistics and without, with one exception: q10 prunes 2 candidates instead of 3. Better estimates change the costs, and changed costs change which candidates lose early. It is a small thing to have noticed, but it is the only place in this paper where an estimate visibly changed what the search *did* rather than only what it believed.

Pruning at q4 discards 25 of 63 candidates by branch-and-bound before they are fully costed. That is where the 74 µs comes from: not from being clever per candidate, but from not costing the ones that already lost.

---

6. Dissection: q9, line by line, across three planners

```
WITH s AS (SELECT dept_id, AVG(salary) AS a FROM emp GROUP BY dept_id)
SELECT e.name FROM emp e JOIN s ON e.dept_id = s.dept_id
 WHERE e.salary > s.a;
```

Moderately complex on purpose: it has a CTE that must be inlined, an aggregate over 20,000 rows into 50 groups, a self-join of `emp` against that aggregate, an equi-key (`dept_id`), and a **non-equi residual** (`salary > a`) that cannot be a hash key. Actual result: **10,000 rows**.

What each system produced

PostgreSQL 18 (61.5 µs parse+plan, 22 µs planner only):

```
Hash Join  (rows=6667)
  Hash Cond: (e.dept_id = emp.dept_id)
  Join Filter: ((e.salary)::numeric > (avg(emp.salary)))
  ->  Seq Scan on emp e  (rows=20000)
  ->  Hash  (rows=50)
        ->  HashAggregate  (rows=50)  Group Key: emp.dept_id
              ->  Seq Scan on emp  (rows=20000)
```

DuckDB (1302 µs):

```
PROJECTION (~663 rows)
└── HASH_JOIN  INNER
      Conditions: dept_id = dept_id
                  CAST(salary AS DOUBLE) > a
    ├── SEQ_SCAN emp (~20,000)
    └── PERFECT_HASH_GROUP_BY  Groups: dept_id  Aggregates: avg(salary)  (~45)
          └── SEQ_SCAN emp (~20,000)
```

DB25 (22.7 µs front end, 12.4 µs planner):

```
(Project exprs=[(col #0)]
  (Filter pred=(> (col #2) (col #4))
    (HashJoin kind=inner build=right keys=[(L#1 R#0)]
      (SeqScan table=emp fmt=row)
      (Project exprs=[(col #0) (col #1)]
        (HashAggregate keys=[(col #0)] aggs=[(AVG (col #1))]
          (SeqScan table=emp fmt=row))))))
```

Where they agree

All three make the same four decisions, and they are the decisions that matter:

1. **Inline the CTE.** None of the three materialises `s`. PostgreSQL 12+ inlines a non-recursive CTE referenced once; DuckDB and DB25 never materialise it in the first place. A materialised CTE here would cost a temporary relation for 50 rows.
2. **Hash-aggregate, not sort-aggregate.** Nothing downstream wants `dept_id`-order, so paying `n log n` to get grouping for free loses. DB25 reaches this by costing both: `StreamingAggregate` is a candidate in the same memo group, priced at `streaming_aggregate_row` (0.5) against `hash_aggregate_row` (1.1) — cheaper per row — and loses anyway, because it would have to enforce a Sort of 20,000 rows to become applicable and the search charges it for the enforcer it causes.
3. **Hash join, building the 50-row side.** All three build the small side. In DB25 this is not a rule — `build_right` is a *costed candidate*, both orientations are enumerated, and the comparison `50 × hash_build_row + 20000`

`× hash_probe_row` against `20000 × hash_build_row + 50 × hash_probe_row` settles it. Both orientations are among q9's 11 candidates, and one of them loses.

4. `salary > a` **is not a join key.** All three recognise that an inequality cannot be hashed and re-check it per candidate row.

That four-way agreement is the actual headline of this section. A young planner agreeing with PostgreSQL and DuckDB on every structural decision of a query with a CTE, an aggregate, a join and a residual is the thing worth reporting — far more than any microsecond count.

Where they differ, and why

Placement of the residual. PostgreSQL folds `salary > a` into the join as a `Join Filter`. DuckDB folds it in as a second join condition. DB25 emits a separate `Filter` node above the join.

Semantically identical; the difference is one operator boundary. Folded in, the predicate is evaluated inside the probe loop on each matched pair before the row is materialised. Hoisted out, the row is produced and then discarded. For an inner join with a 50-row build side this is a small constant either way, and DB25's cost model prices the Filter honestly (`filter_row` per input row) so it is not free in the comparison — but it is a genuine difference, and the folded-in form is the better one. It is a known simplification of DB25's join lowering, not an oversight: the `residual` conjuncts *are* carried on the join node (`residual=[...]` in the IR, see q10's lateral join in Appendix B), so the machinery exists; the inner-join path currently prefers to hoist. That is a lowering change, not a model change.

Estimates. Here the three diverge sharply, and the reason is instructive:

	estimate	actual	implied selectivity for `salary > a`
actual	—	10,000	0.50
PostgreSQL	6,667		0.333
DB25	**6,666**		**0.333**
DuckDB	663		0.033

Every system gets the join right — 20,000 rows, each employee matching exactly one department average — and then applies a **guess** to the inequality. DB25 and PostgreSQL make the same guess, `1/3`, the Selinger default for an inequality against a value that is not a constant. DuckDB's works out around `0.033`.

Nobody has a statistic for this. Comparing a column against a per-group aggregate *of that same column* needs a correlation between `salary` and its own group mean, which no histogram carries and no `ANALYZE` computes. The true answer of 0.5 is what you would get from knowing the distribution is roughly symmetric within each department, and that is an assumption none of the three is entitled to make.

So this is the query where the statistics work stops, and it stops for everyone at the same place. Earlier in this project DB25 answered 2,000 here — its flat tenth — against PostgreSQL's 6,667. It now answers 6,666. That improvement is **not** a statistic being consulted; it is a better default, reached by splitting the fallback by predicate class so an unestimable inequality gets a third rather than a tenth. Worth separating from q1 and q5, where real statistics really are doing the work. We did not solve q9. We guessed better, and we guessed the same as PostgreSQL.

That is the honest lesson of the dissection, and it is section 7.

---

7. Where DB25 is worst, stated plainly

Two queries, and PostgreSQL is wrong on both by the same amounts.

query	actual	DB25	PostgreSQL	why
q6 having	50	17	17	the `HAVING` filters an **aggregate's output**, which has no statistics
q9 CTE + join	10,000	6,666	6,667	`salary > s.a` compares a column against **its own group mean**

Both are the same shape: a predicate whose selectivity is not a property of any single column, so no per-column statistic answers it. Both systems fall back to the Selinger third, and both land in the same wrong place. That is not a defect either catalog can fix by holding more numbers; it needs a correlation nobody computes.

Everything else in the cardinality table is exact or within 3% — the two- and six-way joins are the widest at 2.04%, and the other six are exact or nearly so.

What is still a guess, named

The estimator falls back by predicate class rather than by one flat constant, and those fallbacks remain guesses:

	value	used when
equality	0.005	`col = <non-constant>`, or no ndv for the column
inequality	0.3333	a range that cannot be interpolated — q6 and q9 above
unknown	0.1	a `LIKE`, a function call, a `CASE`

They are named, they live in one place, and their comments say they are guesses. They are also the textbook values rather than invented ones, which is why DB25 and PostgreSQL agree where both are guessing.

What the statistics deliberately do not include

No histogram, and no most-common-value list. `ndv` plus a numeric range answers the predicates that dominate these queries, and answers them *exactly* where the data is uniform — which this fixture is. On skewed data a range interpolation is wrong in a bounded, explainable way, and an MCV list is the next thing to add. It is not here because a histogram nobody has measured against is a second set of invented numbers wearing a better hat.

Also still true: q9 shows DB25 hoisting a residual out of a join where both mature systems fold it into the join condition. Semantically identical, one operator boundary apart, and the folded-in form is the better one. The machinery exists — the IR carries `residual` on the join node — and the inner-join lowering currently prefers to hoist. A lowering change, not a model change, and recorded rather than buried.

---

8. What this does not show

- **No execution.** Everything here is about choosing a plan, nothing about running one. A planner that chooses well and an engine that executes well are different problems and only one of them is measured.

- **One fixture, ten queries, one machine.** Six tables and 192,000 rows. Every join is a clean foreign key. Real schemas have composite keys, skew, correlation, NULLs and outer joins in combinations this fixture does not contain — and all of those are where estimates go wrong.
- **In-process versus not.** DB25 pays no client protocol and no catalog round trip. Part of its margin is architectural, not algorithmic.
- **No statistics maintenance cost, and no statistics pipeline.** PostgreSQL and DuckDB paid for `ANALYZE`. DB25's statistics were handed to it: the catalog is caller-seeded, and nothing here measures, samples or refreshes anything. When there is an engine, `ANALYZE` has to exist and will cost what it costs. This paper measures a planner given correct statistics, not a system that maintains them.
- **DuckDB is being measured outside its design point.** Its planning cost is built to be amortised over an executor's seconds. Reading its 1.4 ms as a weakness would be exactly the kind of meaningless number this paper opened by disclaiming.
- **Absolute microseconds are a property of the host, not of the software.** An earlier draft was measured on a 2.80 GHz instance and reported 74 µs for q4; the same binary on a 2.10 GHz instance reported 58.6, because the faster-labelled host was busier. Only figures taken in the same session against each other mean anything, which is why all three systems here were re-measured back to back rather than carried over.
- **And this host is bimodal, not merely noisy.** Nine repeats of one binary on one query give two clusters — q1 is 6.4 µs or 10.8 µs — with nothing in the program selecting which. Pinning to a core narrows it and does not remove it. That is why section 3 reports ranges and withdraws every per-query timing claim smaller than 2×, and why an A/B measurement there nearly reported a 32% effect that was two samples of two modes. Ratios *within* a run are unaffected, which is why the stage shares are stated confidently and the absolutes are not.
- **PostgreSQL's estimates move slightly too.** `ANALYZE` samples, so re-running it shifts the multi-way join estimate between 99,491 and 100,000 and the semi-join between 19,690 and 19,739. Small, but it is the reason the q-error figures are given to two decimals and not three — and the reason 1.16 against 1.17 is read as a tie.
- **These numbers move.** Taken on 2026-10-14 against `db25@673e71c`. They will be wrong by the next increment, and that is the intended failure mode — the numbers exist to be falsified, not framed.

---

9. Conclusion

Three things came out of this.

The search machinery is sound. Where DB25 has the information it needs, its output matches two mature planners decision for decision: same join algorithms, same build sides, same aggregate strategies, same CTE inlining, the same refusal to hash an inequality. It does that on the hardest join in the set in about 56 µs of planning, exploring 63 candidates and pruning 25, against PostgreSQL's 158 — the one per-query timing claim this host can support.

Simple statistics go a long way. Four numbers per column — distinct values, null fraction, numeric range, a uniqueness flag — took the geometric-mean q-error from 3.09× to 1.16×, against PostgreSQL's 1.17× with histograms, MCV lists and correlation. Eight of the ten queries are now exact or within 3%. No histogram was needed to get there, and saying so is more useful than the tie itself.

What is left is not a statistics problem. The two remaining errors are a filter over an aggregate and a column compared against its own group mean. Neither is answered by any per-column statistic, both systems fall back to the

same textbook default, and both are wrong by the same amount. The next real improvement on this corpus would come from correlation statistics that nobody computes — or from admitting that some predicates are guesses and saying so, which is what both planners already do.

A closing note on method, since it cost more time than the code did. This paper twice reported a result that was not there: an earlier draft's timing wins, which did not survive a 50% run-to-run spread, and a 32% statistics speedup that was two samples of a bimodal distribution. Both were caught by checking a quantity that *could not* have changed — parse time, candidate counts — rather than by doubting the number directly. The estimates in section 4 carry none of that risk, because they are exactly reproducible. That is a reason to prefer them as the measure of a planner, quite apart from their being the measure that decides the plan.

Neither of which is a benchmark. It was fun, and it cost rather more than an afternoon.

Appendix A — reproducing

Everything is in `docs/bench/`:

file	what it is
`schema.sql` · `data.sql`	the six tables and their 192,000 rows
`queries.txt`	the ten queries, `id|label|sql`
`db25_bench.cpp`	DB25 timing: median of N warm iterations, catalog construction hoisted out of the loop
`db25_plans.cpp`	DB25 plan shape, estimated rows, memo groups, candidates, goals, pruned
`db25_stages.cpp`	the same pipeline with a clock between each stage: parse, analyze+bind, logical optimize, physical lowering
`pg_wall.py`	PostgreSQL both ways: `EXPLAIN (SUMMARY ON)` xN for planner time, and wall time for N `EXPLAIN`s minus N baseline statements for parse+plan
`others_bench.py`	PostgreSQL planner time and DuckDB in-process `EXPLAIN`, minus the same baseline

```
# Where the cluster is. The two PostgreSQL harnesses read these, so the same
# scripts point at any build without being edited.
export PSQL_BIN=/opt/pg18/bin/psql PGHOST=/tmp/pgs18 PGPORT=55433

$PSQL_BIN -h $PGHOST -p $PGPORT -U postgres -d bench -f docs/bench/schema.sql
$PSQL_BIN -h $PGHOST -p $PGPORT -U postgres -d bench -f docs/bench/data.sql

# DB25_STATS=1 seeds the column statistics; without it the estimator falls
# back to its flat constants, which is how the 3.09x row in section 4 was taken.
DB25_STATS=1 ./db25_stages docs/bench/queries.txt 400   # timing, stage by stage
DB25_STATS=1 ./db25_plans  docs/bench/queries.txt       # plans and estimates
./db25_bench docs/bench/queries.txt 500                 # timing, one region
python3 docs/bench/pg_wall.py 200            # PostgreSQL, both boundaries
python3 docs/bench/others_bench.py 300       # PostgreSQL planner + DuckDB
```

The DuckDB fixture is built in-process by `others_bench.py` from the same row generators, so the two engines see identical data rather than merely similar data.

`pg_wall.py` builds one `psql -c` body holding N copies of the query, which is why N is 200 rather than larger: at N=400 the reference query overflows the argument list.

Repeat counts, and why they are what they are. DB25's timings are medians of nine process invocations, PostgreSQL's of five, DuckDB's of three. Nine is not caution, it is the minimum that makes a bimodal distribution visible: fewer repeats return a median that is simply whichever mode was sampled more, which is how an earlier draft of section 4 came to report a 32% effect that did not exist. The estimates need no repeats at all — they are deterministic, and nine runs of `db25_plans` produce byte-identical output.

Appendix B — DB25's plan for q10

The reference query, showing the lateral join, the correlation reference, the window and the enforced sorts:

```
(Limit limit=10
  (Sort by=[#3 asc]
    (Project exprs=[(col #1) (col #3) (col #5) (col #8) (col #7)]
      (Window fns=[(RANK :over [:partition [(col #2)] :order [(col #3) desc]])]
        (Sort by=[#2 asc #3 desc]
          (Filter pred=(> (col #3) (col #5))
            (NestedLoopJoin kind=leftlateral keys=[] residual=[(lit true)]
              (Exchange to=single
                (HashJoin kind=inner build=right keys=[(L#2 R#0)]
                  (SeqScan table=emp fmt=row)
                  (Project exprs=[(col #0) (col #1) (col #2)]
                    (HashAggregate keys=[(col #0)]
                                   aggs=[(AVG (col #1)) (COUNT)]
                      (SeqScan table=emp fmt=row)))))
              (Project exprs=[(col #0)]
                (StreamingAggregate keys=[] aggs=[(SUM (col #1))]
                  (Filter pred=(= (col #0) (outer d1 #0))
                    (SeqScan table=orders fmt=row)))))))))))
```

Four things to notice. The lateral is a `NestedLoopJoin` because it *must* be — its right side carries an `(outer d1 #0)` reference to the left row, so no build-once algorithm is applicable. The scalar `SUM` is a `StreamingAggregate` with no keys, which is exactly one row out regardless of input. The two `Sort`s are enforcers placed by the search, not written by the query: one to give the window its partition-and-order requirement, one for the `ORDER BY rnk` that the window's own output does not provide. And the root `Limit` clamps the estimate to 10 — the only query in the set where DB25, PostgreSQL and DuckDB were asked for a number that the SQL itself determines, and the only one DuckDB missed (5,000).

DB25 is an experiment in building a SQL front end where every design decision is falsifiable. The gap register (`docs/gap-register.md`) records what is known to be wrong, including everything in section 7.